Lab Programs

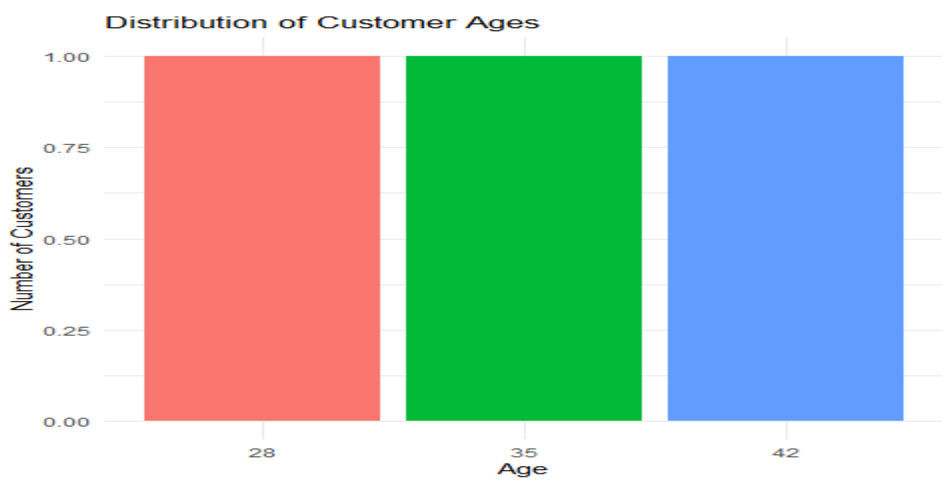
SET 7- Customer Demographics Analysis

7A. Bar Chart

Program:

```
customer_data <- data.frame(
CustomerID=c(1,2,3),
Age=c(28,35,42),
Gender=c("Female","Male","Female"),
Income=c(50000,60000,75000))
ggplot(customer_data,
aes(x=factor(Age),
fill=factor(Age)))+
geom_bar()+
labs(title="Distribution of Customer Ages",
x="Age",
y="Number of Customers")+
theme_minimal()+
theme(legend.position="none")
```

Output:

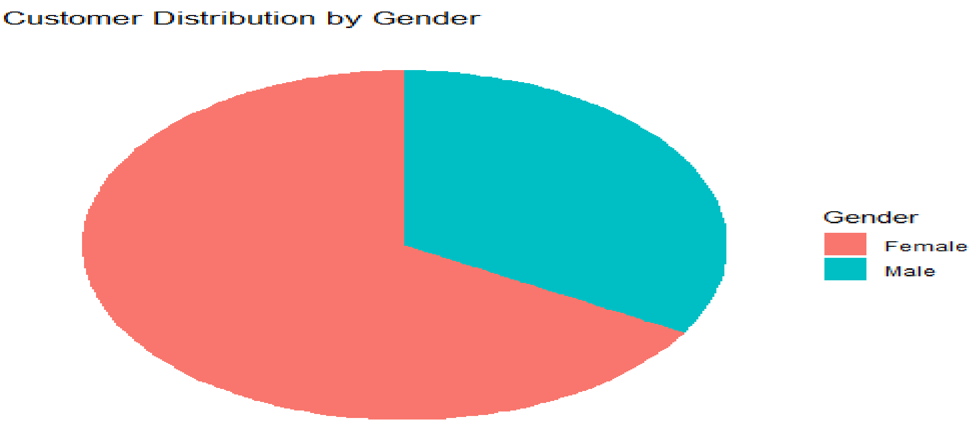


7B. Pie Chart

Program:

```
gender_data <- data.frame(table(customer_data$Gender))
colnames(gender_data) <- c("Gender","Count")
ggplot(gender_data,
aes(x="",y=Count,fill=Gender))+
geom_bar(stat="identity",width=1)+
coord_polar("y")+
labs(title="Customer Distribution by Gender")+
theme_void()
```

Output:



7C. Table

Program:

```
kable(
customer_data,
caption="Customer Demographics Information")
```

Output:

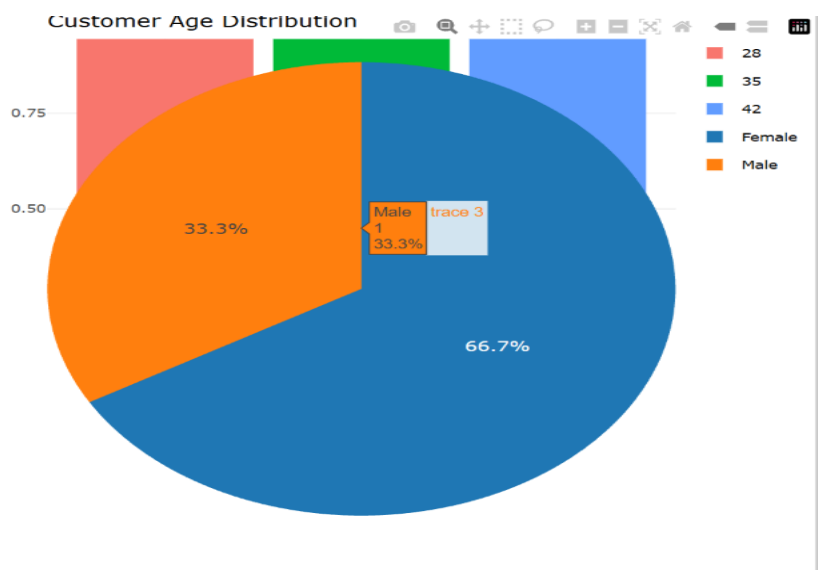
```
Table: Customer Demographics Information

| CustomerID| Age|Gender | Income|
|----------:|---:|:------|------:|
|          1|  28|Female |  50000|
|          2|  35|Male   |  60000|
|          3|  42|Female |  75000|
> 
```

7D. Combined Interactive Dashboard

Program:

```
library(plotly)
bar_plot <- ggplotly(
  ggplot(customer_data,
         aes(x=factor(Age), fill=factor(Age))) +
    geom_bar() +
    labs(title="Customer Age Distribution") +
    theme_minimal() +
    theme(legend.position="none"))
gender_data <- data.frame(
  Gender=c("Female","Male"),
  Count=c(2,1))
pie_plot <- plot_ly(
  gender_data,
  labels=~Gender,
  values=~Count,
  type="pie")
dashboard <- subplot(bar_plot, pie_plot, nrows=2)
dashboard
```

Output:



SET 8- Employee Performance Analysis

8A. Line Chart

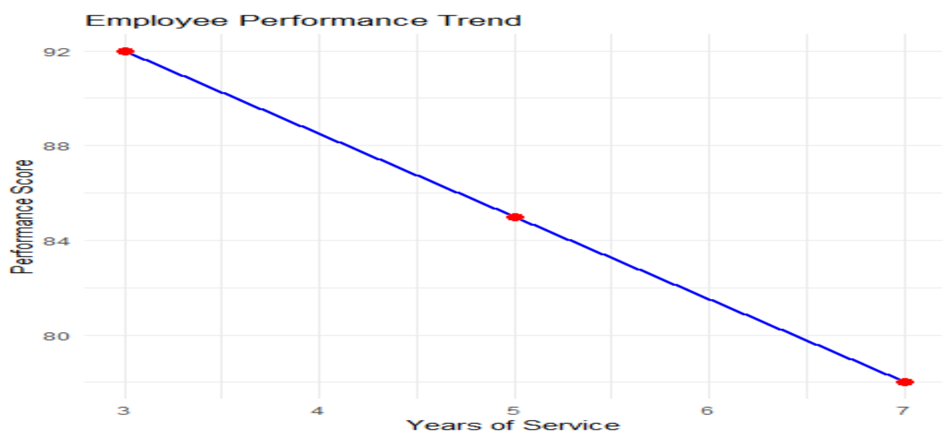
Step 1: Install Packages (Run Only Once)

```
install.packages("ggplot2")
install.packages("plotly")
install.packages("knitr")
```

Program:

```
employee_data <- data.frame(
EmployeeID=c(1,2,3),
Department=c("Sales","HR","Marketing"),
YearsOfService=c(5,3,7),
PerformanceScore=c(85,92,78))
ggplot(employee_data,
aes(x=YearsOfService,
y=PerformanceScore,
group=1))+
geom_line(color="blue",size=1)+
geom_point(color="red",size=3)+
labs(title="Employee Performance Trend",
x="Years of Service",
y="Performance Score")+
theme_minimal()
```

Output:

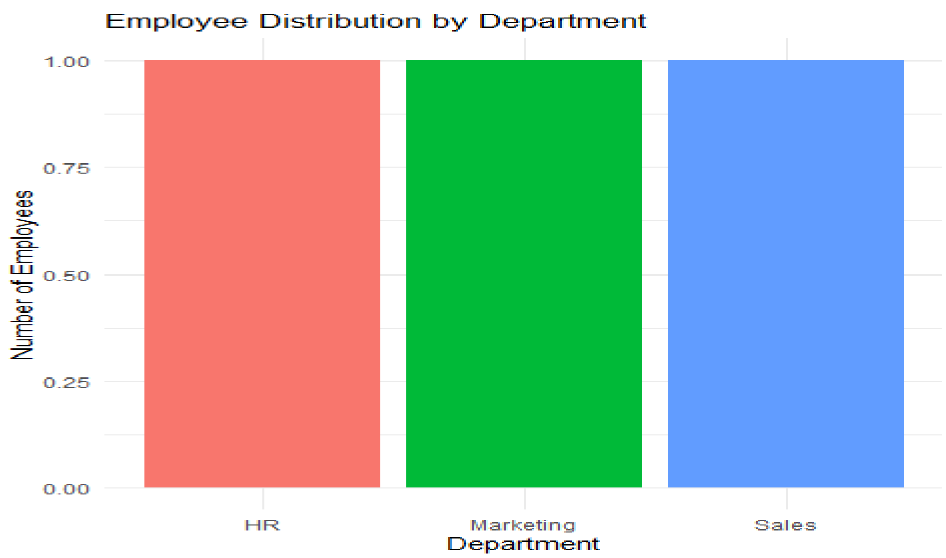


8B. Bar Chart

Program:

```
ggplot(employee_data,
aes(x=Department,
fill=Department))+
geom_bar()+
labs(title="Employee Distribution by Department",
x="Department",
y="Number of Employees")+
theme_minimal()+
theme(legend.position="none")
```

Output:



8C. Table

Program:

```
kable(
employee_data,
caption="Employee Performance Data")
```

Output:

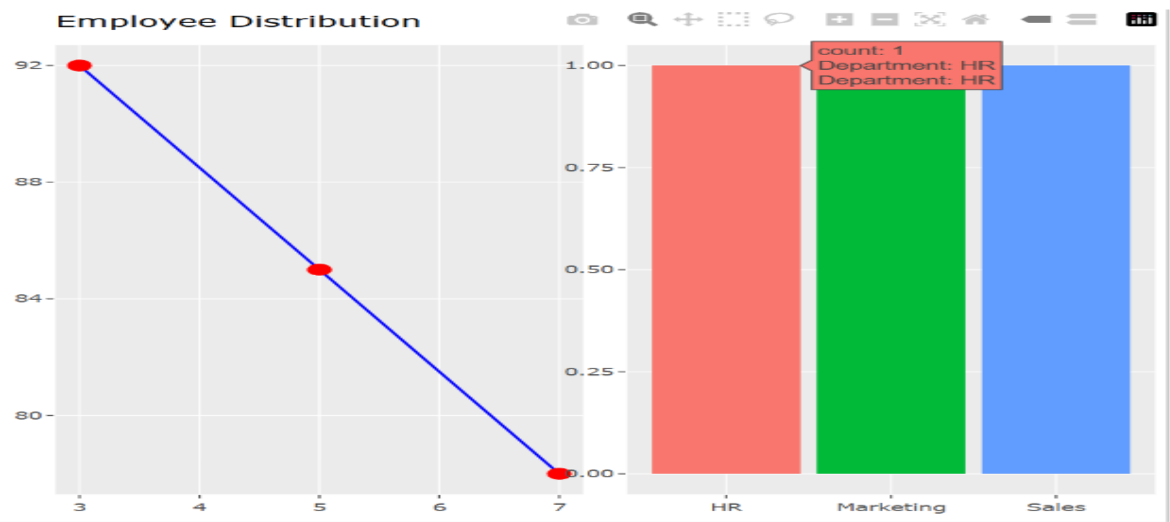
```
Table: Employee Performance Data

| EmployeeID|Department | YearsOfService| PerformanceScore|
|----------:|:----------|--------------:|----------------:|
|          1|Sales      |              5|               85|
|          2|HR         |              3|               92|
|          3|Marketing  |              7|               78|
> |
```

8D. Interactive Dashboard

Program:

```
library(plotly)
line_plot <- ggplotly(
ggplot(employee_data,
aes(x=YearsOfService,
y=PerformanceScore, group=1))+
geom_line(color="blue")+
geom_point(color="red",size=3)+
labs(title="Employee Performance Trend"))
bar_plot <- ggplotly(
ggplot(employee_data,
aes(x=Department,
fill=Department))+
geom_bar()+
labs(title="Employee Distribution")+
theme(legend.position="none")) dashboard <- subplot(
line_plot,bar_plot, nrows=1,
shareX=FALSE,
shareY=FALSE)
dashboard
```

Output:



Set 9- Online_Learning_Activity.csv

9A. Import Dataset, Histogram & Boxplot

Program:

```
data <- read.csv(file.choose())
```

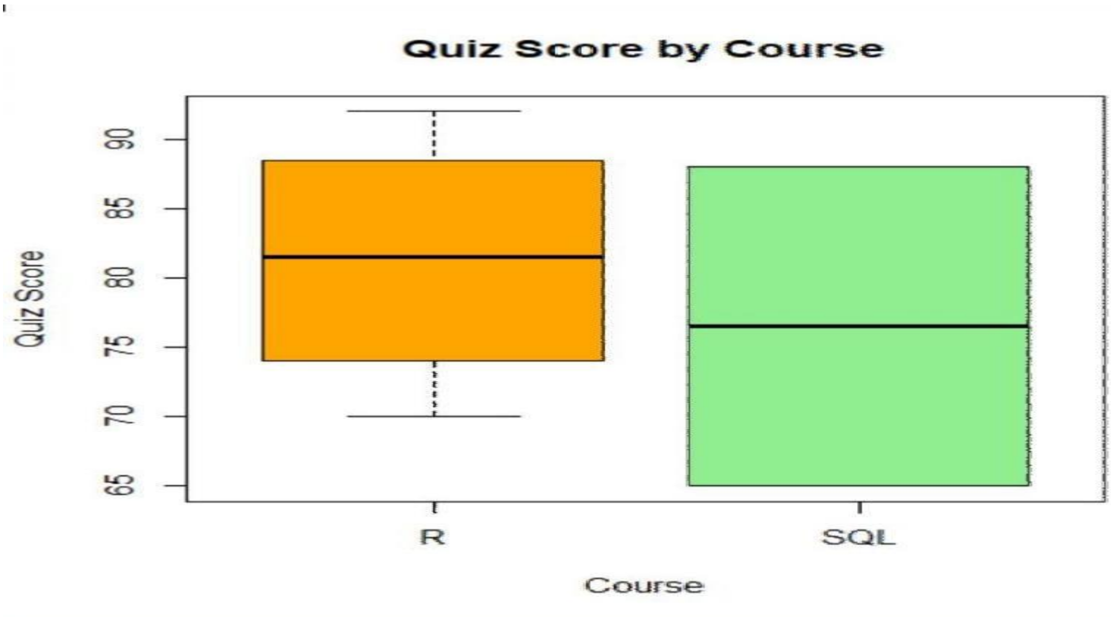
Histogram:

```
ggplot(data,aes(x=Quiz_Score))+
geom_histogram(binwidth=5,fill="skyblue",color="black")+
labs(title="Distribution of Quiz Scores",
x="Quiz Score", y="Number of Students")+
theme_minimal()
```

Boxplot:

```
ggplot(data,aes(x=Course,
y=Quiz_Score, fill=Course))+
geom_boxplot()+
labs(title="Quiz Score by Course",
x="Course",
y="Quiz Score")+
theme_minimal()
```

Output:

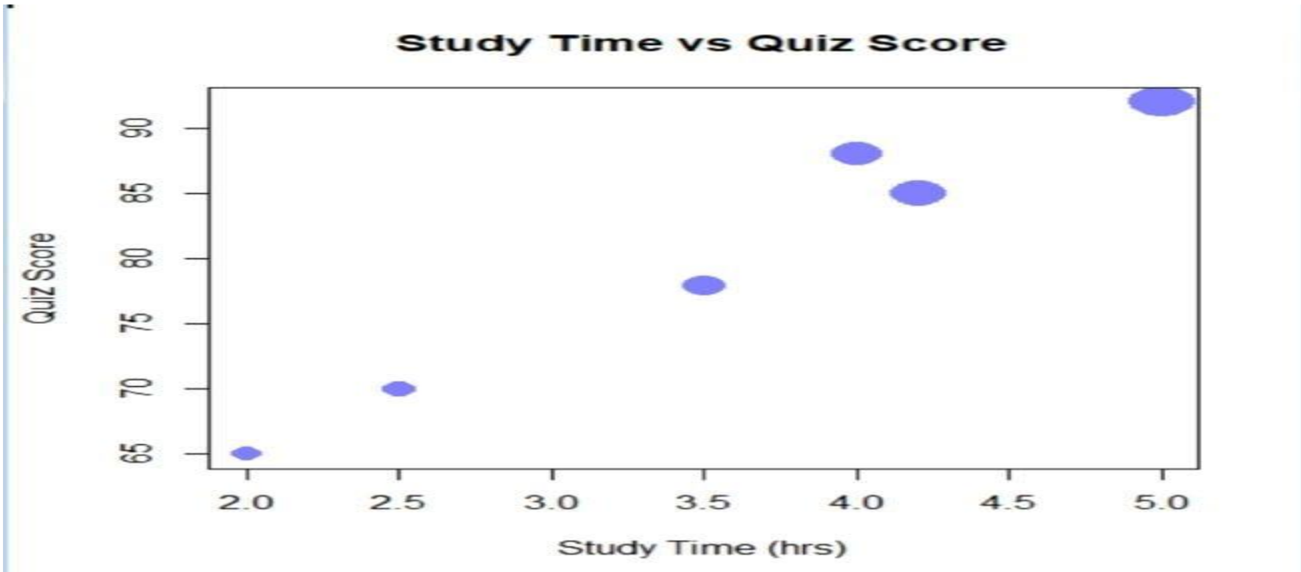


9B. Scatter Plot with Bubble Size

Program:

```
ggplot(data,
aes(x=Study_Time,
y=Quiz_Score,
size=Videos_Watched,
color=Course))+
geom_point(alpha=0.6)+
labs(title="Study Time vs Quiz Score",
x="Study Time (hrs)",
y="Quiz Score",
size="Videos Watched")+
theme_minimal()
```

Output:



9C. Date Conversion, Monthly Average & Line Chart

Program:

Convert Date

```
data$Login_Date <- as.Date(data$Login_Date)
```

Extract Month

```
data$Month <- format(data$Login_Date,"%Y-%m")
```

Average Quiz Score Per Month

```
monthly <- data %>%
group_by(Month) %>%
summarise(Avg_Quiz=mean(Quiz_Score))
```

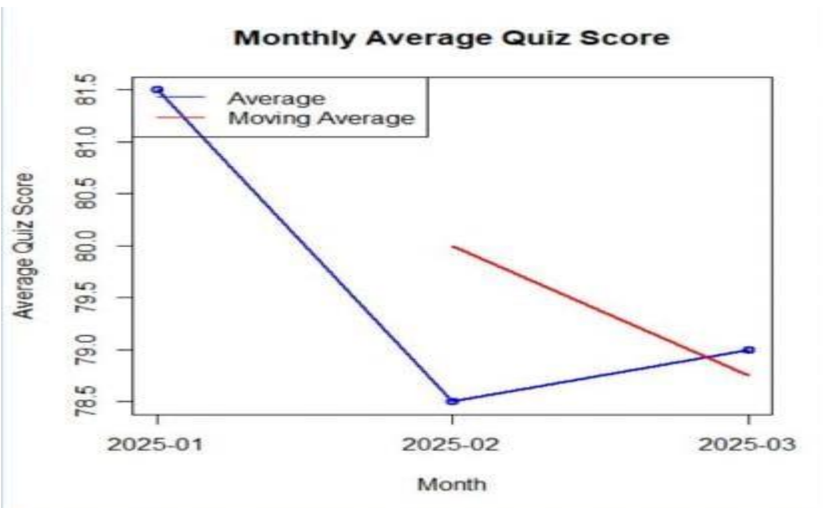
Moving Average

```
monthly$MovingAvg <- rollmean(
monthly$Avg_Quiz,
k=2,fill=NA,
align="right")
```

Line Chart

```
ggplot(monthly,aes(x=Month,y=Avg_Quiz,group=1))+
geom_line(color="blue",size=1)+
geom_point(size=3,color="red")+
geom_line(aes(y=MovingAvg),
color="green",
linetype="dashed",
size=1)+
labs(title="Average Monthly Quiz Score",
x="Month",
y="Average Quiz Score")+
theme_minimal()
```

Output:



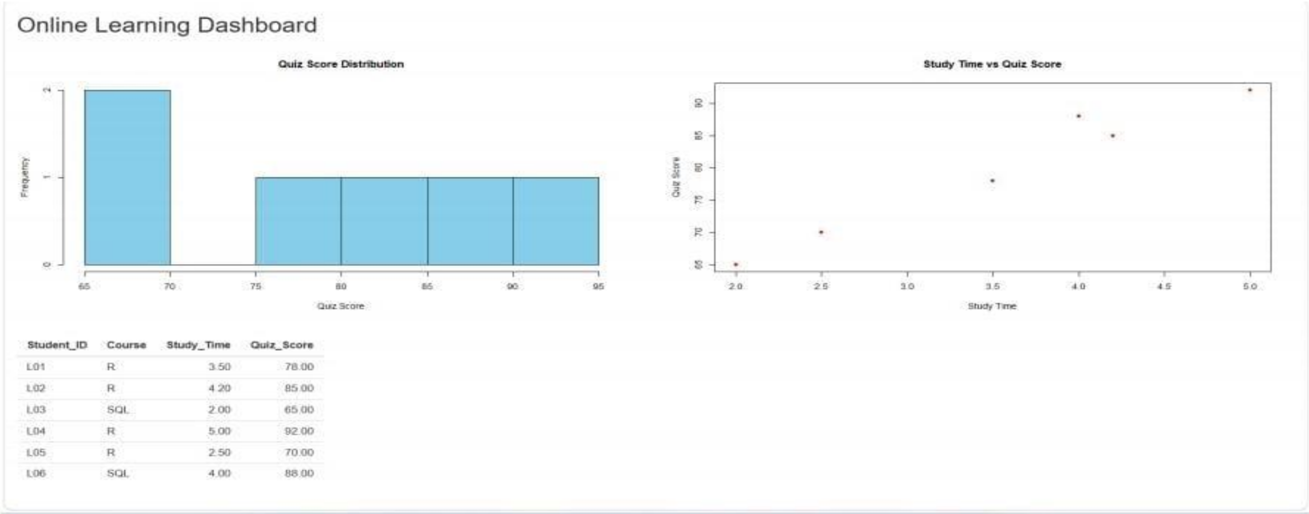
9D. Interactive Dashboard

Program:

```
library(shiny)
data <- data.frame(
Student_ID=c("L01","L02","L03","L04","L05","L06"),
Gender=c("Male","Female","Male","Female","Male","Female"),
Course=c("R","R","SQL","R","R","SQL"),
Study_Time=c(3.5,4.2,2.0,5.0,2.5,4.0),
Quiz_Score=c(78,85,65,92,70,88)
)
ui <- fluidPage(
titlePanel("Online Learning Dashboard"),
sidebarLayout(
sidebarPanel(
selectInput("gender",
"Gender",
choices=c("All",unique(data$Gender))),
selectInput("course",
"Course",
choices=c("All",unique(data$Course))) ),
mainPanel(
fluidRow(
column(6,
plotOutput("bar")),
column(6,
plotOutput("scatter"))
) ) )
)
server <- function(input,output){
output$bar <- renderPlot({
```

```
d <- data
if(input$gender!="All")
d <- subset(d,Gender==input$gender)
if(input$course!="All")
d <- subset(d,Course==input$course)
avg <- tapply(d$Quiz_Score,d$Course,mean)
barplot(avg,
col="skyblue",
main="Average Quiz Score by Course",
ylab="Quiz Score")
})
output$scatter <- renderPlot({
d <- data
if(input$gender!="All")
d <- subset(d,Gender==input$gender)
if(input$course!="All")
d <- subset(d,Course==input$course)
plot(d$Study_Time,
d$Quiz_Score,
pch=16,
col="red",
xlab="Study Time",
ylab="Quiz Score",
main="Study Time vs Quiz Score")
})
}
shinyApp(ui,server)
```

Output:



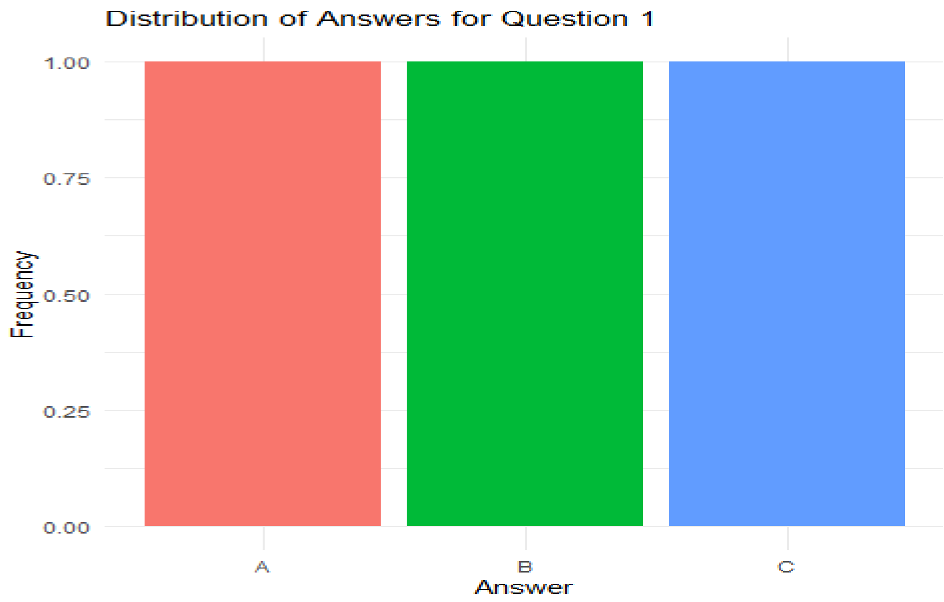
Set 10: Survey Responses Analysis

10A. Grouped Bar Chart

Program:

```
survey_data <- data.frame(
SurveyID=c(1,2,3),
Question1=c("A","B","C"),
Question2=c("B","A","A"),
Question3=c("C","D","B"))
q1_count <- as.data.frame(table(survey_data$Question1))
colnames(q1_count) <- c("Answer","Count")
ggplot(q1_count,
aes(x=Answer,
y=Count,
fill=Answer))+
geom_bar(stat="identity")+
labs(title="Distribution of Answers for Question 1",
x="Answer",
y="Frequency")+
theme_minimal()+
theme(legend.position="none")
```

Output:

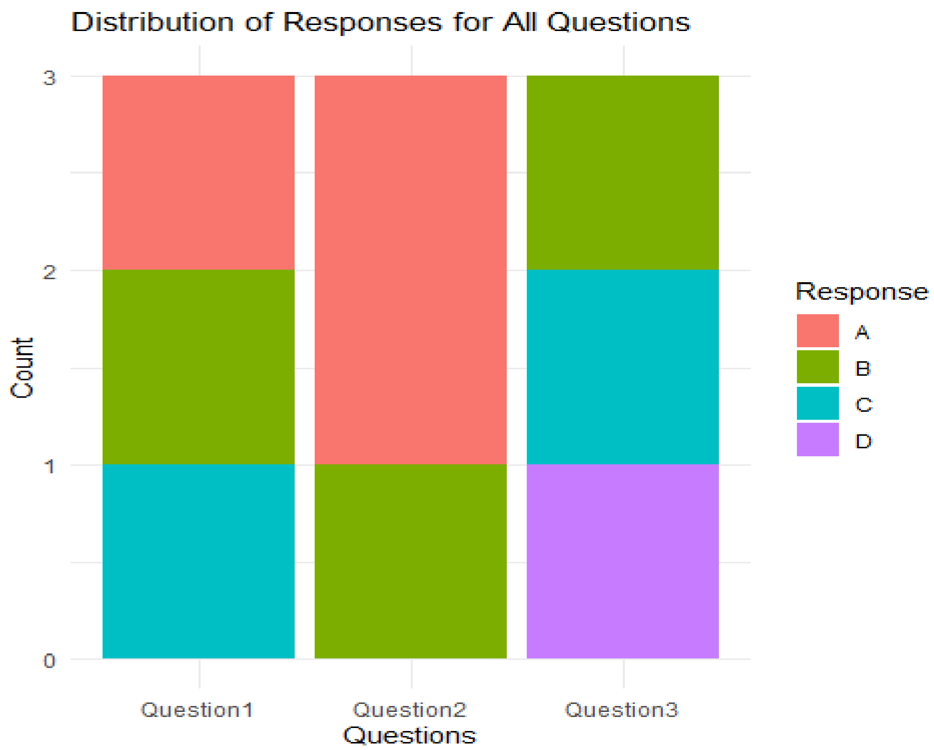


10 B. Stacked Bar Chart

Program:

```
survey_long <- melt(
survey_data,
id.vars="SurveyID",
variable.name="Question",
value.name="Response")
ggplot(survey_long,
aes(x=Question,
fill=Response))+
geom_bar()+
labs(title="Distribution of Responses for All Questions",
x="Questions",
y="Count",
fill="Response")+
theme_minimal()
```

Output:



10C. Table

Program:

kable(

survey_data,

caption="Survey Response Data"

)

Output:

```
Table: Survey Response Data

| SurveyID|Question1 |Question2 |Question3 |
|--------:|:---------|:---------|:---------|
|        1|A         |B         |C         |
|        2|B         |A         |D         |
|        3|C         |A         |B         |
> |
```

10D. Interactive Dashboard (R)

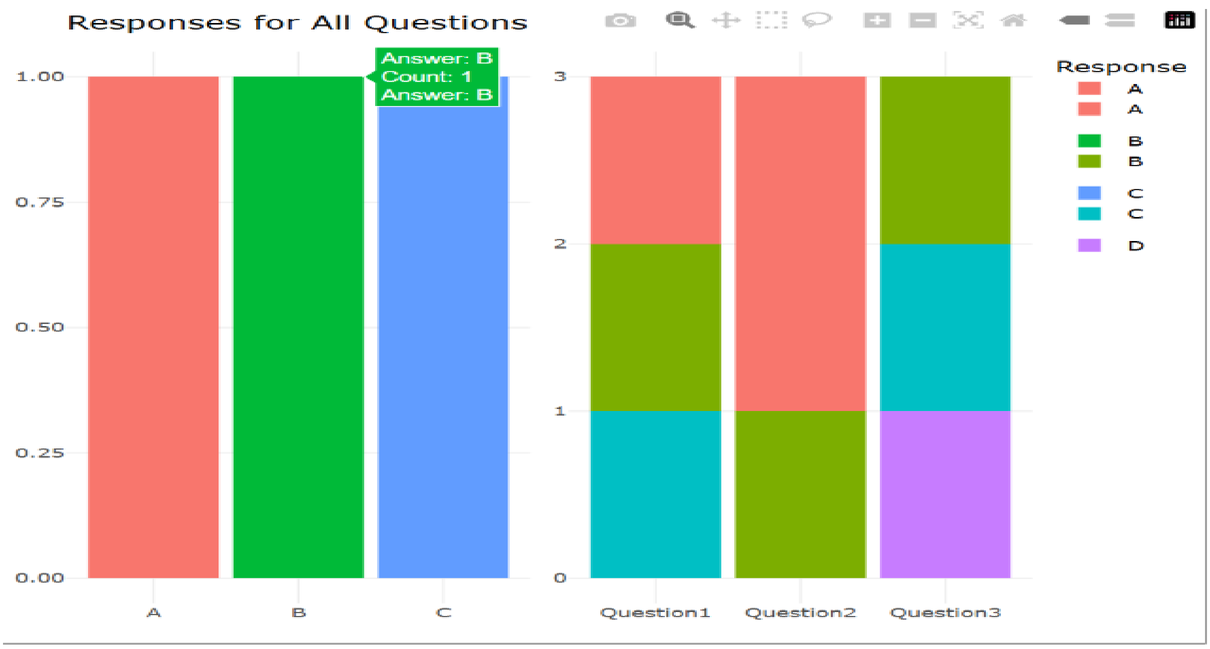
Program:

bar_plot <- ggplotly(

ggplot(q1_count,

```
aes(x=Answer,
y=Count,
fill=Answer))+
geom_bar(stat="identity")+
labs(title="Question 1 Responses")+
theme_minimal()+
theme(legend.position="none"))
stacked_plot <- ggplotly(
ggplot(survey_long,
aes(x=Question,
fill=Response))+
geom_bar()+
labs(title="Responses for All Questions")+
theme_minimal())
dashboard <- subplot(bar_plot,
stacked_plot, nrows=1,
shareX=FALSE,
shareY=FALSE)
dashboard
```

Output:



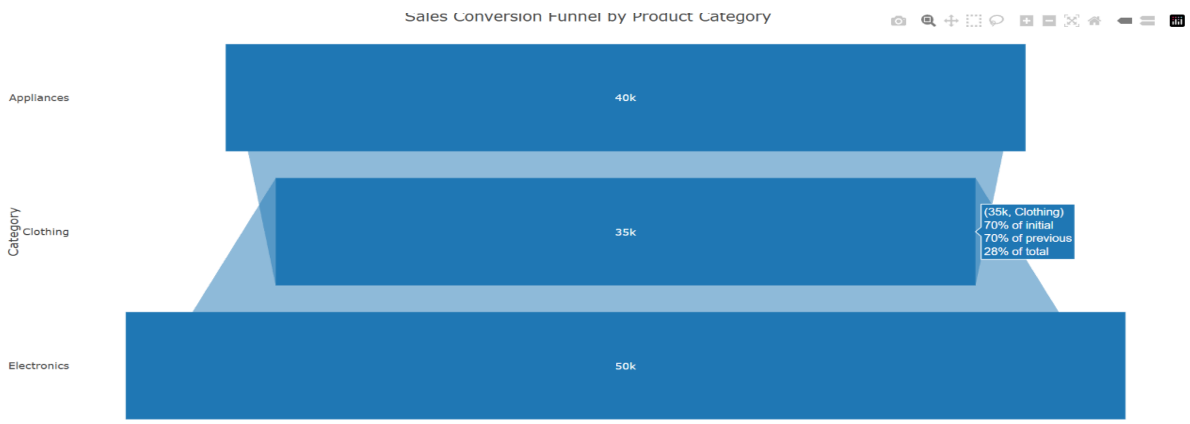
SET 11- Product Category Analysis

11 A. Funnel Chart

Program:

```
product_data <- data.frame(
Category=c("Electronics","Clothing","Appliances"),
Sales=c(50000,35000,40000))
plot_ly( product_data,
type="funnel",
y=~Category,
x=~Sales) %>%
layout(title="Sales Conversion Funnel by Product Category")
```

Output:



11 B. Table

Program:

```
kable(
product_data,
caption="Product Category Sales")
```

Output:

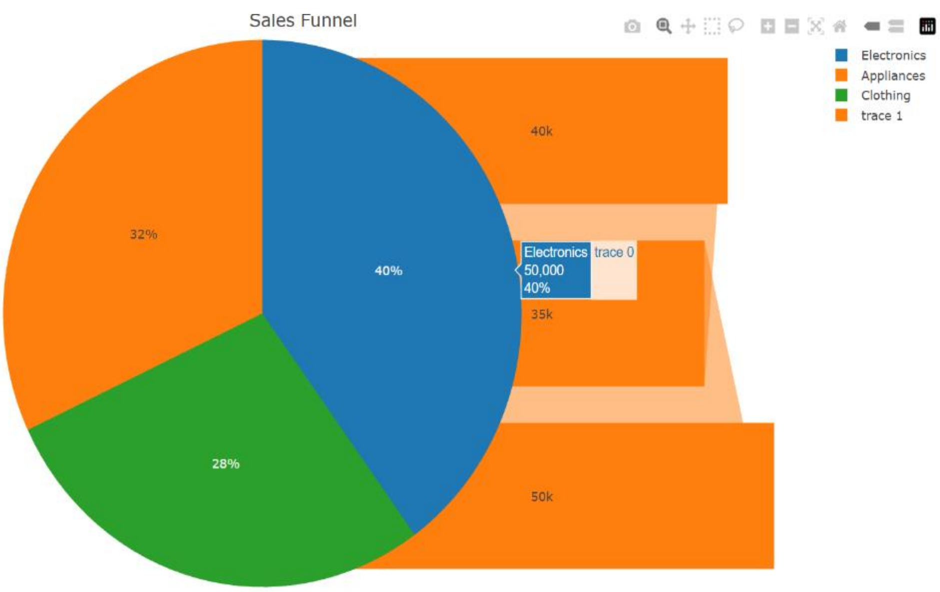
```
Table: Product Category Sales

|Category    | Sales|
|:-----------|-----:|
|Electronics | 50000|
|Clothing    | 35000|
|Appliances  | 40000|
>
```

11 C. Interactive Dashboard (R)

Program:

```
pie_plot <- plot_ly(
product_data,
labels=~Category,
values=~Sales,
type="pie") %>%
layout(title="Sales Distribution")
funnel_plot <- plot_ly(
product_data,
type="funnel",
y=~Category,
x=~Sales) %>%
layout(title="Sales Funnel")
dashboard <- subplot(
pie_plot,
funnel_plot,
nrows=1)
dashboard
```

Output:

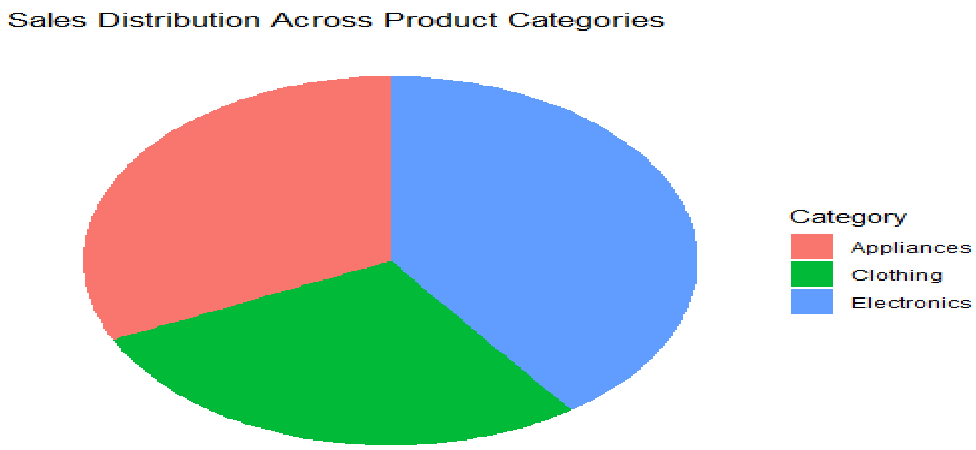


11 D. Pie Chart

Program:

```
ggplot(product_data,
aes(x="",
y=Sales,
fill=Category))+
geom_bar(stat="identity",width=1)+
coord_polar("y")+
labs(title="Sales Distribution Across Product Categories")+
theme_void()
```

Output:



SET 12- Website Traffic

12 A. Line Chart

Program:

```
website_data <- data.frame(
Date=as.Date(c("2023-01-01","2023-01-02","2023-01-03")),
PageViews=c(1500,1600,1400),
CTR=c(2.3,2.7,2.0))
ggplot(website_data,
aes(x=Date,y=PageViews,group=1))+
```

geom_line(color="blue",size=1)+

geom_point(color="red",size=3)+

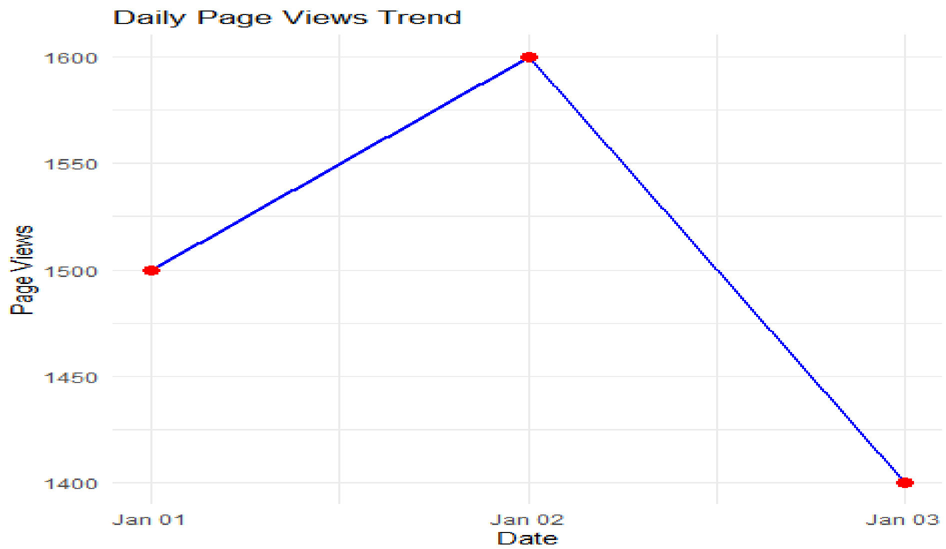
labs(title="Daily Page Views Trend",

x="Date",

y="Page Views")+

theme_minimal()

Output:



12 B. Bar Chart (Top Days by Click-through Rate)

Program:

top_ctr <- website_data[order(-website_data$CTR),]

ggplot(top_ctr,

aes(x=reorder(as.character(Date),CTR),

y=CTR,

fill=CTR))+

geom_bar(stat="identity")+

coord_flip()+

labs(title="Top Days by Click-through Rate",

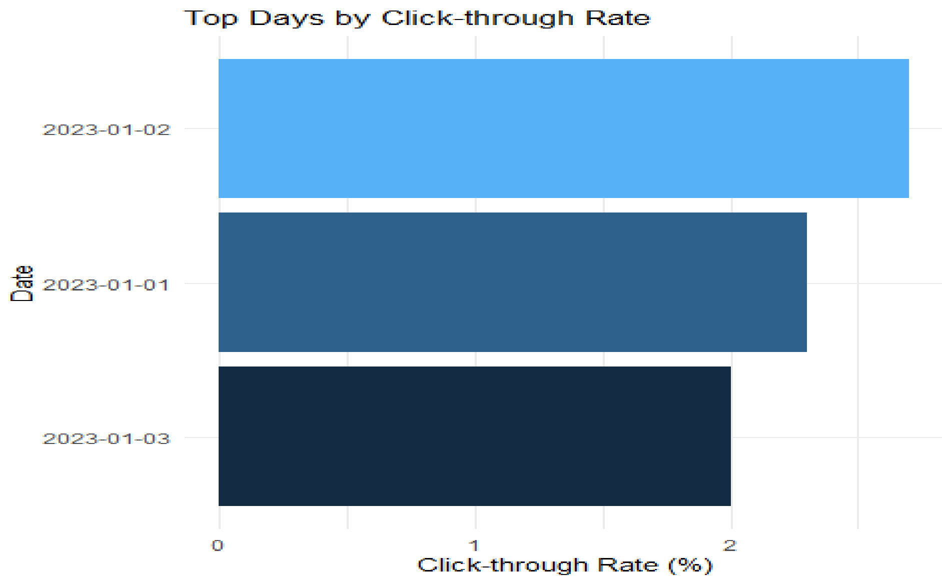
x="Date",

y="Click-through Rate (%)")+

theme_minimal()+

theme(legend.position="none")

Output:



12 C. Table

Program:

kable(

website_data,

caption="Website Traffic Data")

Output:

```
Table: Website Traffic Data

|Date        | PageViews| CTR|
|:-----------|---------:|---:|
|2023-01-01  |      1500| 2.3|
|2023-01-02  |      1600| 2.7|
|2023-01-03  |      1400| 2.0|
>
```

12 D. Interactive Dashboard

Program:

line_plot <- ggplotly(

ggplot(website_data,

aes(x=Date,y=PageViews,group=1))+

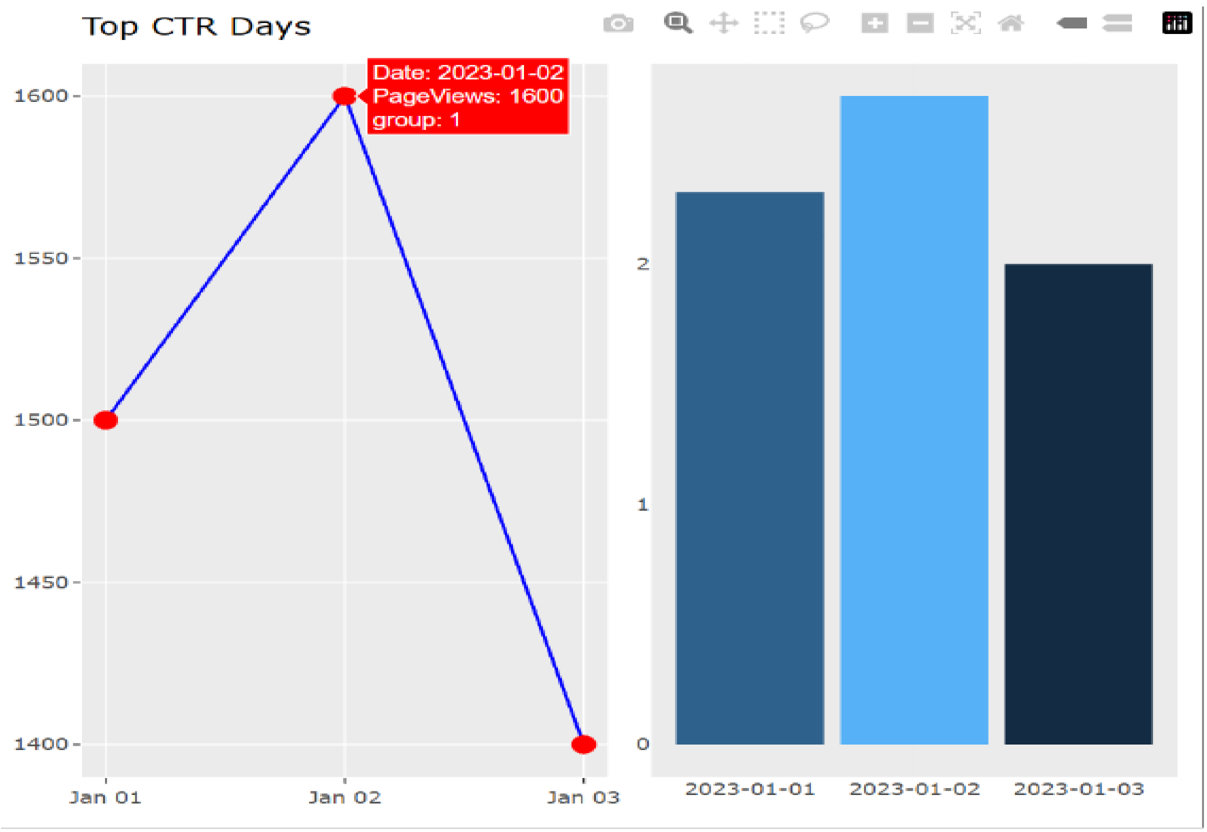
geom_line(color="blue")+

geom_point(color="red",size=3)+

labs(title="Daily Page Views"))

```
bar_plot <- ggplotly(
ggplot(top_ctr,
aes(x=as.character(Date),
y=CTR, fill=CTR))+
geom_bar(stat="identity")+
labs(title="Top CTR Days")+
theme_minimal()+
theme(legend.position="none"))
dashboard <- subplot(
line_plot,
bar_plot, nrows=1,
shareX=FALSE,
shareY=FALSE)
dashboard
```

Output:



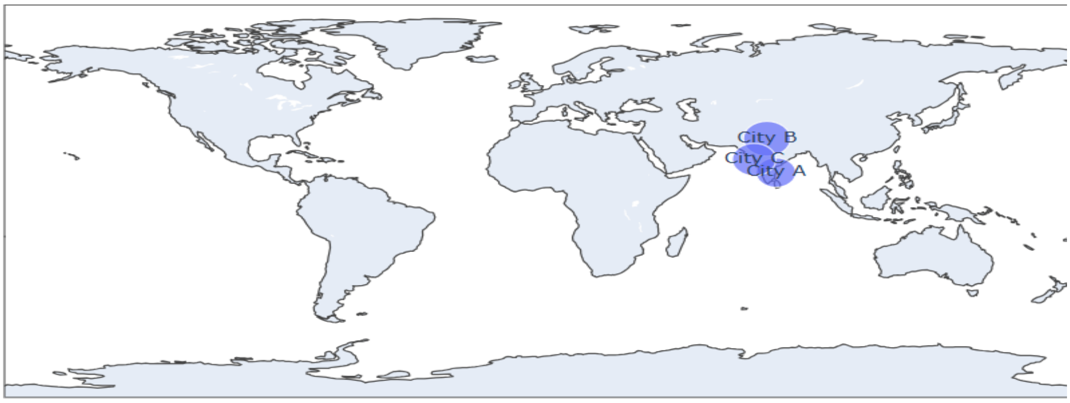
SET 13- Geographic Data

13 A. Map Chart

Program:

```
import pandas as pd
import plotly.express as px
data = pd.DataFrame({
    "City":["City A","City B","City C"],
    "Population":[500000,700000,600000],
    "Temperature":[75,68,80],
    "Elevation":[1000,800,1200],
    "Latitude":[13.08,28.61,19.07],
    "Longitude":[80.27,77.21,72.87]
})
fig = px.scatter_geo(
    data,
    lat="Latitude",
    lon="Longitude",
    text="City",
    size="Population",
    title="Distribution of Cities"
)
fig.update_layout(
    geo=dict(showland=True),
    title_x=0.5
)
fig.show()
```

Output:

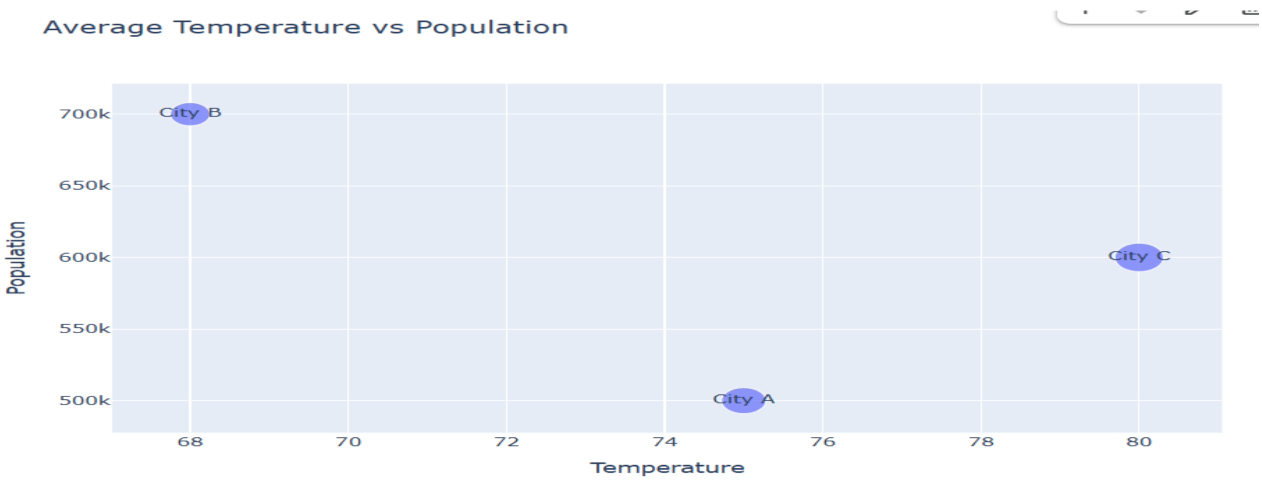


13 B. Scatter plot

Program:

```
import plotly.express as px
fig = px.scatter(
    data,
    x="Temperature",
    y="Population",
    text="City",
    size="Elevation",
    title="Average Temperature vs Population"
)
fig.show()
print("Insight:")
print("City B has the highest population with the lowest temperature, while City C has the highest temperature.")
```

Output:



13 C. Table

Program:

```
from IPython.display import display
display(data)
```

Output:

	City	Population	Temperature	Elevation	Latitude	Longitude
0	City A	500000	75	1000	13.08	80.27
1	City B	700000	68	800	28.61	77.21
2	City C	600000	80	1200	19.07	72.87

13 D. Tableau Dashboard

Program:

```
data.to_csv("Geographic_Data.csv", index=False)
```

Output:

Gemini Geographic_Data.csv

1 to 3 of 3 entries Filter

City	Population	Temperature	Elevation	Latitude	Long
City A	500000	75	1000	13.08	80.27
City B	700000	68	800	28.61	77.21
City C	600000	80	1200	19.07	72.87

SET 14- Survey Results

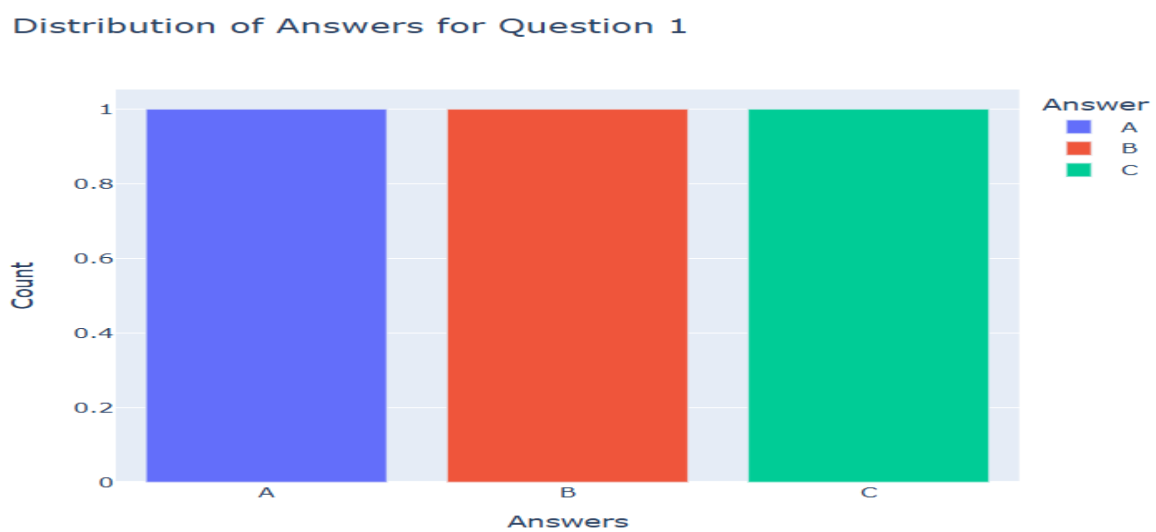
14 A. Stacked Bar Chart

Program:

```
import pandas as pd
import plotly.express as px
survey = pd.DataFrame({
    "Respondent":[1,2,3],
    "Question 1":["A","B","C"],
    "Question 2":["B","A","A"],
    "Question 3":["C","D","B"]
```

```
})
count = survey["Question 1"].value_counts().reset_index()
count.columns = ["Answer","Count"]
fig = px.bar(
    count,
    x="Answer",
    y="Count",
    color="Answer",
    title="Distribution of Answers for Question 1"
)
fig.update_layout(
    xaxis_title="Answers",
    yaxis_title="Count"
)
fig.show()
```

Output:



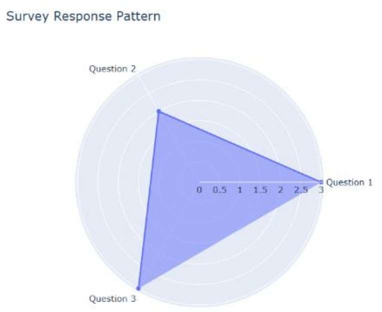
14 B. Radar chart

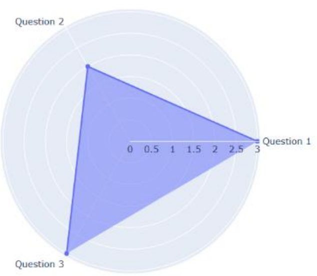
Program:

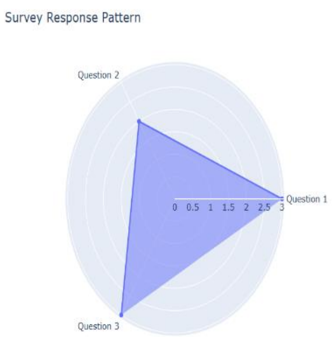
```
import plotly.graph_objects as go
responses = [
    survey["Question 1"].nunique(),
```

```
    survey["Question 2"].nunique(),
    survey["Question 3"].nunique()
]
fig = go.Figure()
fig.add_trace(go.Scatterpolar(
    r=responses,
    theta=["Question 1","Question 2","Question 3"],
    fill='toself',
    name='Responses'
))
fig.update_layout(
    polar=dict(radialaxis=dict(visible=True)),
    title="Survey Response Pattern"
)
fig.show()
```

Output:







14 C. Table

Program:

```
from IPython.display import display
display(survey)
```

Output:

	Respondent	Question 1	Question 2	Question 3
0	1	A	B	C
1	2	B	A	D
2	3	C	A	B

14 D. Tableau Dashboard

Program:

```
survey.to_csv("Survey_Results.csv", index=False)
```

Output:

Gemini Survey_Results.csv

1 to 3 of 3 entries Filter

Respondent	Question 1	Question 2	Question 3
1	A	B	C
2	B	A	D
3	C	A	B

Show 10 per page

SET 15- Product Sales Analysis

15 A. Grouped Bar Chart

Program:

```
import pandas as pd
import plotly.express as px
sales = pd.DataFrame({
    "Product":["Product A","Product B","Product C"],
    "January":[2000,1500,1200],
    "February":[2200,1800,1400],
    "March":[2400,1600,1100]
})
sales_long = sales.melt(id_vars="Product",
                        var_name="Month",
                        value_name="Sales")
fig = px.bar(
    sales_long,
    x="Product",
    y="Sales",
    color="Month",
    barmode="group",
    title="Monthly Product Sales - First Quarter"
```

)

fig.update_layout(

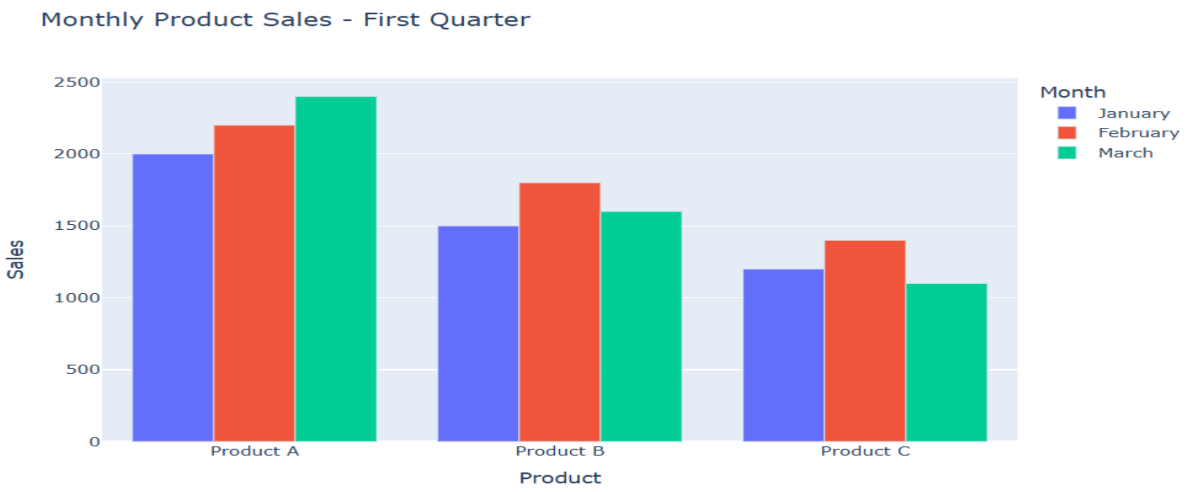
    xaxis_title="Product",

    yaxis_title="Sales"

)

fig.show()

Output:



15 B. Stacked Area Chart

Program:

fig = px.area(

    sales_long,

    x="Month",

    y="Sales",

    color="Product",

    title="Overall Sales Trend"

)

fig.update_layout(

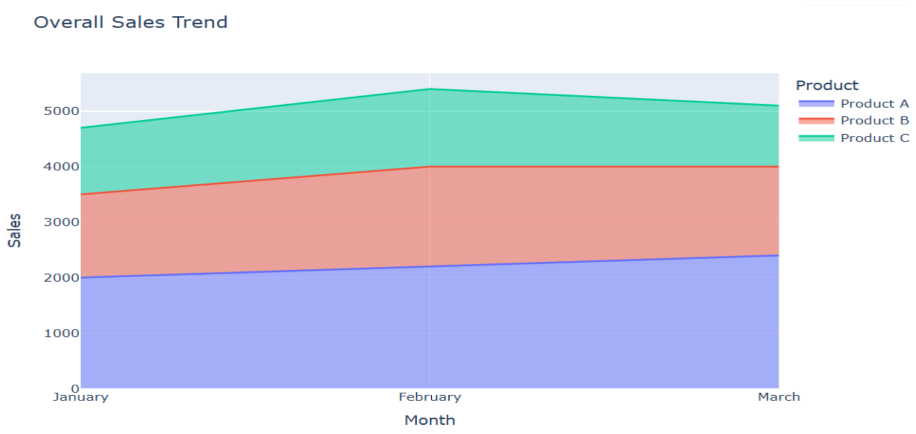
    xaxis_title="Month",

    yaxis_title="Sales"

)

fig.show()

Output:



15 C. Table

Program:

from IPython.display import display

display(sales)

Output:

	Product	January	February	March
0	Product A	2000	2200	2400
1	Product B	1500	1800	1600
2	Product C	1200	1400	1100

15 D. Tableau Dashboard

Program:

sales.to_csv("Monthly_Product_Sales.csv", index=False)

Output:

Monthly_Product_Sales.csv × Gemi

1 to 3 of 3 entries Filter

Product	January	February	March
Product A	2000	2200	2400
Product B	1500	1800	1600
Product C	1200	1400	1100

Show 10 per page